Edwin Maraví
emaravi@cjavaperu.com

CJAVA

CJAVA

siempre para apoyarte

Visión

Poder aportar al desarrollo del País usando tecnología Java.

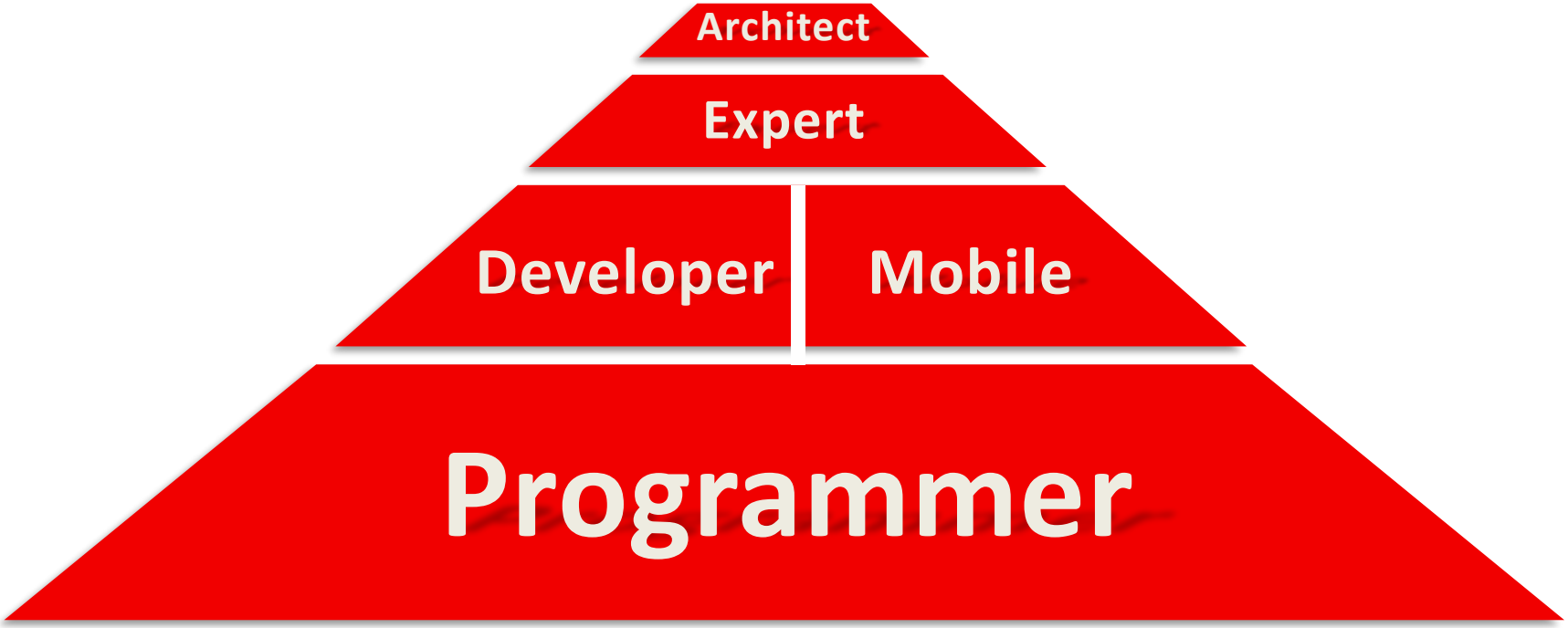
CJAVA
siempre para apoyarte

Quienes Somos

Somos una organización orientada a desarrollar, capacitar e investigar tecnología JAVA a través de un prestigioso staff de profesionales a nivel nacional.

CJAVA
siempre para apoyarte

La Carrera JAVA



Consultas con Hibernate

Edwin Maravi

CJAVA
siempre para apoyarte

Temas

- Consultas Hibernate
- HQL y JPQL
- Criteria API
- QBE (Query ByExample)

CJAVA
siempre para apoyarte

Tópicos

- ¿Qué es Criteria Query?
- ¿Cómo usar Criteria Query?
- Paginación
- Restricciones
- Ordenamientos
- Agregación de funciones
- Modos de Fetch
- Query By Example (QBE)

CJAVA
siempre para apoyarte

Tres maneras de obtener data en Hibernate

- **Criteria query API**
  - La manera mas sencilla de obtener data.
  - Basado en puro lenguaje Java

- **Hibernate Query Language (HQL)**
- **Native SQL query**

CJAVA
siempre para apoyarte

Criteria Query

- **Utiliza un conjunto de objetos Java para construir consultas.**
  - En lugar del lenguaje de consulta
- **Soporta Query By Example (QBE)**
  - Realización de una consulta, proporcionando un objeto ejemplo que contienen propiedades que necesitan ser recuperados
- **Soporta metodos de agregación(from Hibernate 3)**
  - **Count**
  - **AVG**
  - **Max**
  - **Min**

CJAVA
siempre para apoyarte

Como usar Criteria

- **Crear un objeto *org.hibernate.Criteria via el método createCriteria() de la Session***
  - Pasar el class del objeto persistente o el nombre de la clase al método *createCriteria()*
- **Llamar al método *list() del objeto Criteria***

```
// Obtener todas las instancias de Empleado
Criteria crit = sess.createCriteria(Empleado.class);
List results = crit.list();
```

CJAVA
siempre para apoyarte

Paginación con Criteria

Hibernate se encarga de la paginación

– Recuperando un número fijo de objetos

Dos métodos de la clase Criteria

– *setFirstResult() – establece la primera fila en el resultado*
– *setMaxResults() – número de registros a obtener*

```
Criteria crit = sess.createCriteria(Empleado.class);
crit.setFirstResult(2);
crit.setMaxResults(50);
List results = crit.list();
```

CJAVA
siempre para apoyarte

Consultas con Restrictions

- Usado para obtener objetos selectivamente
  - Example: Empleados que son mayores a 20.

- Añadir restricciones al objeto Criteria con el método **add()**
  - El método *add() del objeto Criteria toma un objeto* ***org.hibernate.criterion.Criterion*** *que representa una restricción individual.*

- Tú puedes tener más de una restricción para un Criteria Query.

CJAVA
siempre para apoyarte

Metodos de la Clase Restrictions

- Restrictions.eq(“name”, ”Shin”)
- Restrictions.ne(“name”, ”NoName”)
- Restrictions.like(“name”, “Sa%”)
- Restrictions.ilike(“name”, “sa%”)
- Restrictions.isNull(“name”);
- Restrictions.gt(“price”,new Double(30.0))
- Restrictions.between(“age”, new Integer(2), new Integer(10))
- Restrictions.or(criterion1, criterion2)
- Restrictions.disjunction()

Ordenando con Restrictions

Puedes ordenar los resultados usando *org.hibernate.criterion.Order*

```
List cats = sess.createCriteria(Cat.class)
    .add( Restrictions.like("name", "F%")
    .addOrder( Order.asc("name") )
    .addOrder( Order.desc("age") )
    .setMaxResults(50)
    .list();
```

CJAVA
siempre para apoyarte

Projections y Agregates

- ***rowCount()***
- ***avg(String propertyName)***
  - promedio del valor de una propiedad
- ***count(String propertyName)***
  - número de veces
- ***countDistinct(String propertyName)***
  - número de únicos valores que contiene una propiedad
- ***max(String propertyName)***
- ***min(String propertyName)***
- ***sum(String propertyName)***
  - suma de los valores de una propiedad

CJAVA
siempre para apoyarte

Projections

- **Projections.rowCount()**

```
Criteria crit = sess.createCriteria(Person.class);
crit.setProjection( Projections.rowCount() );
List results = crit.list();
```

Multiple Projections

- **Projections.projectionList()**

```
Criteria crit = sess.createCriteria(Product.class);
ProjectionList projectList = Projections.projectionList();
projectList.add(Projections.avg("price"));
projectList.add(Projections.sum("price"));
crit.setProjection( projectList );
List results = crit.list();
```

Fecth Modes

- FetchMode.DEFAULT
- FetchMode.JOIN
- FetchMode.SELECT

CJAVA
siempre para apoyarte

Query By EXample

- **Use Example.create() para crear una restricción**

```
 // Obtiene empleados vía by Example
 Criteria crit = sess.createCriteria(Empleado.class);
 Empleado empl = new Empleado();
 empl.setName("Juan");
Example exampleRestriction = Example.create(empl);
crit.add( exampleRestriction );
List results = crit.list();
```

CJAVA
siempre para apoyarte

HQL

- Qué es HQL?
- la cláusula “from”
- Asociaciones y join
- cláusula “select”
- query Polymorphic
- cláusula “where”

CJAVA
siempre para apoyarte

HQL

- HQL es full orientada a objetos
- Las querys son case-insensitive, excepto para los nombres de las clases Java y propiedades

Clausula “from”

- **Retorna todas las instancias de la clase eg.Cat.**

from eg.Cat

- **Usualmento no necesitas colocar el nombre completo de la clase**

  from Cat es lo mismo que from eg.Cat

- **La mayoría de veces tu necesitarás asignar un alias para referirte a Cat en otras partes de la query**

from Cat as cat (“cat” es el alias)

from Cat cat (“as” puede ser omitido)

CJAVA
siempre para apoyarte

Asociacianones y Joins

- También podemos asignar alias a las entidades asociadas, e incluso a los elementos de una colección de valores, utilizando un **join**

```
from Cat as cat
  inner join cat.mate as mate
  left outer join cat.kittens as kitten
```

Tipos de join

- inner join
- left outer join
- right outer join

CJAVA
siempre para apoyarte

Cláusula Select

- **La cláusula select escoge qué objetos y propiedades se van a devolver**

```
select mate
  from Cat as cat
      inner join cat.mate as mate
```

Cláusula select

- **Las querys pueden retornar propiedades de otras propiedades**

```
select cat.name from DomesticCat cat
        where cat.name like 'fri%'
select cust.name.firstName from Customer as cust
```

- **Las querys pueden retornar múltibles objetos y/o propiedades como un array de tipo Object[]**

```
select mother, offspr, mate.name
from DomesticCat as mother
   inner join mother.mate as mate
   left outer join mother.kittens as offspr
```

CJAVA
siempre para apoyarte

Links de referencia

- Web oficial de hiberante
  - https://www.hibernate.org/
- API de hibernate 3
  - https://www.hibernate.org/hib_docs/v3/api/
- Hibernate Reference Documentantion
  - http://docs.jboss.org/hibernate/stable/core/reference/en/html/

CJAVA

siempre para apoyarte

Gracias